

# Asistente de comparación de productos

## Abstract

Este proyecto consiste en el desarrollo, mediante el apoyo de la genAI, de una aplicación inteligente para la comparación de productos mediante el uso de modelos de lenguaje (LLM). La solución permite al usuario introducir varios productos, definir criterios de evaluación y obtener un análisis estructurado en formato JSON con puntuaciones, ventajas, desventajas y una recomendación final. La aplicación integra el modelo *Llama 3.3 70B Versatile (Meta)* a través de la API de Groq, que utiliza técnicas de *prompt engineering* para garantizar respuestas coherentes, parseables y orientadas a la toma de decisiones. Además, se implementa una interfaz visual basada en Streamlit que facilita la interpretación de resultados mediante tarjetas, barras de puntuación y resúmenes comparativos. El proyecto demuestra cómo mediante el uso de interfaces intuitivas y modelos generativos avanzados se puede automatizar tareas complejas de análisis comparativo de forma eficiente y escalable.

## Introducción

En el contexto actual del comercio digital, los usuarios se enfrentan continuamente a la necesidad de comparar productos antes de realizar una compra. La gran cantidad de información disponible, junto con opiniones contradictorias y especificaciones técnicas complejas, dificulta la toma de decisiones objetiva y rápida.

El reto principal de este proyecto consiste en diseñar una aplicación capaz de generar comparativas automáticas y estructuradas entre distintos productos utilizando inteligencia artificial generativa. La solución debe ser capaz de:

- Analizar múltiples productos simultáneamente.
- Adaptarse dinámicamente a distintos criterios de evaluación.
- Generar resultados claros, coherentes y fáciles de interpretar.
- Mantener un formato estructurado que permita su posterior procesamiento automático.

Para resolver este problema se ha desarrollado una aplicación con técnicas de “vibe coding” basada en Streamlit e integrada con un Large Language Model (LLM) mediante la API de Groq. El sistema utiliza técnicas avanzadas de *prompt engineering* para controlar la estructura y calidad de las respuestas generadas por el modelo.

## Implementación de la aplicación

La aplicación se ha desarrollado con Python y Streamlit como framework principal para la interfaz gráfica. También se ha utilizado HTML y CSS embebido para personalizar la apariencia visual, ya que los componentes nativos de Streamlit no permiten un nivel de personalización suficiente. La arquitectura se divide en tres bloques principales:

1. Construcción y envío de prompts al LLM.
2. Procesamiento y validación de respuestas.
3. Renderizado visual de resultados.

La comunicación con el modelo se realiza mediante el cliente oficial de Groq. La clave API se almacena de forma segura utilizando `st.secrets`, evitando exponer credenciales directamente en el código.

```
client = Groq(api_key=st.secrets["GROQ_API_KEY"])
```

El flujo de la aplicación es el siguiente:

1. El usuario introduce los productos a comparar.
2. Opcionalmente define criterios específicos y contexto adicional.
3. La aplicación construye un prompt estructurado.
4. El prompt se envía al modelo LLM.
5. El modelo devuelve una respuesta exclusivamente en formato JSON.
6. El JSON se parsea y valida.
7. Los resultados se representan visualmente mediante componentes dinámicos.

La función `construir_prompt_usuario(productos, criterios, contexto)` genera dinámicamente el prompt que recibe el modelo. Este inserta automáticamente la lista de productos, añade criterios personalizados proporcionados por el usuario, introduce contexto adicional e incluye instrucciones explícitas para devolver un JSON válido

Si el usuario no define criterios, la aplicación utiliza criterios genéricos por defecto ("Calidad, Precio, Durabilidad, Relación calidad-precio"). Esto garantiza que siempre exista una base consistente de evaluación.

La función `llamar_llm()` gestiona la interacción con Groq, usando el modelo llama-3.3-70b-versatile, con una temperatura de 0.3 y un máximo de 2000 tokens. La temperatura baja se utiliza para reducir la aleatoriedad y aumentar la consistencia estructural de las respuestas. Una vez recibida la respuesta se implementa una limpieza defensiva para eliminar posibles bloques Markdown. Finalmente, el contenido se parsea utilizando:

```
json.loads(raw)
```

Este paso es fundamental para garantizar que la respuesta pueda utilizarse automáticamente dentro de la aplicación.

La interfaz utiliza Streamlit para representar visualmente los resultados. La función `render_puntuacion_barra(puntuacion, max_val=10)` genera barras visuales dinámicas mediante la conversión de puntuación en porcentaje y usando colores dinámicos, siguiendo un código de colores estilo semáforo. Esto mejora considerablemente la interpretación visual de los resultados.

La función `render_producto(producto)` muestra cada producto mediante una tarjeta visual. En estas tarjetas se incluye el nombre del producto, su puntuación global, los criterios evaluados y sus pros/contras.

La información se organiza utilizando columnas de Streamlit, mejorando la experiencia de usuario y la legibilidad.

## Integración con LLM

El modelo seleccionado es llama-3.3-70b-versatile de Meta. Este modelo se utiliza a través de la infraestructura de Groq. La elección del modelo se basa en varios factores:

1. *Capacidad de razonamiento.* El modelo ofrece un buen rendimiento en tareas analíticas y comparativas complejas.
2. *Generación estructurada.* Presenta una alta capacidad para respetar formatos JSON estrictos, requisito fundamental del proyecto.
3. *Velocidad de inferencia.* Groq proporciona tiempos de respuesta muy reducidos, mejorando significativamente la experiencia de usuario.
4. *Relación coste-rendimiento.* El modelo ofrece resultados de alta calidad con costes razonables frente a otras alternativas comerciales.

La comunicación sigue una arquitectura sencilla basada en mensajes:

### *System Prompt*

Establece el comportamiento global del modelo. Ejemplo: "Eres un experto analista de productos..."

- El rol del modelo.
- Restricciones estrictas.
- Estructura exacta del JSON.
- Normas de evaluación.

### *User Prompt*

Contiene la petición concreta: "Compara los siguientes productos...":

- Productos.
- Criterios.
- Contexto.
- Solicitud de comparativ

La respuesta debe seguir exactamente esta estructura:

```
{  
  "productos": [...],  
  "ganador": "...",  
  "justificacion_ganador": "...",  
  "resumen_comparativa": "..."  
}
```

Esto permite automatizar la validación de datos, generar renderizados dinámicos, facilitar la escalabilidad futura e integrarse fácilmente con otras aplicaciones.

## Prompt Engineering

El proyecto incorpora diversas técnicas de *prompt engineering* para aumentar la calidad y consistencia de los resultados.

1. *Role Prompting*. El modelo recibe un rol específico: "Eres un experto analista de productos..."

- Incrementa la coherencia analítica.
- Mejora la objetividad de las comparativas.
- Reduce respuestas genéricas.

2. *Output Structuring*. Se obliga al modelo a responder solamente en JSON: "Responde ÚNICAMENTE con un objeto JSON válido."

- Facilita el parseo automático.
- Reduce errores de formato.
- Mejora la integración backend/frontend.

3. *Restricciones explícitas*. Se añaden múltiples reglas estrictas: "No añadas texto adicional..."

- Minimiza respuestas inesperadas.
- Reduce la necesidad de postprocesado.
- Aumenta la robustez del sistema.

4. *Few-shot implícito*. Aunque no se proporcionan ejemplos completos, la estructura JSON detallada actúa como guía implícita.

- Mejora la precisión estructural.
- Facilita el cumplimiento del esquema esperado.

5. *Temperatura controlada*. En este caso se opta por una temperatura de valor 0.3.

- Reduce la creatividad innecesaria.
- Mejora consistencia.
- Disminuye desviaciones del formato requerido.

6. *Contextualización dinámica*. La función de construcción del prompt adapta automáticamente productos, contexto y criterios.

- Incrementa flexibilidad.
- Permite reutilización para múltiples categorías de productos.

## Conclusión

El desarrollo de esta aplicación demuestra el potencial de los modelos de lenguaje para automatizar tareas complejas de análisis comparativo y asistencia a la toma de decisiones. Mediante la integración de un LLM avanzado con una interfaz visual intuitiva, se consigue transformar información textual en comparativas estructuradas, claras y útiles para el usuario.

Uno de los principales retos del proyecto ha sido garantizar la consistencia y validez del formato de salida generado por el modelo. Para resolverlo, se han aplicado técnicas de *prompt engineering* orientadas a restringir el comportamiento del LLM y asegurar respuestas parseables en JSON.

Mediante Streamlit, Groq y Llama 3.3 se puede obtener una solución rápida, escalable y fácilmente extensible a otros dominios. Además, el proyecto evidencia cómo el diseño adecuado de prompts puede influir directamente en la calidad, estabilidad y utilidad práctica de los resultados generados por una inteligencia artificial.